

V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,

Ajou University, Korea

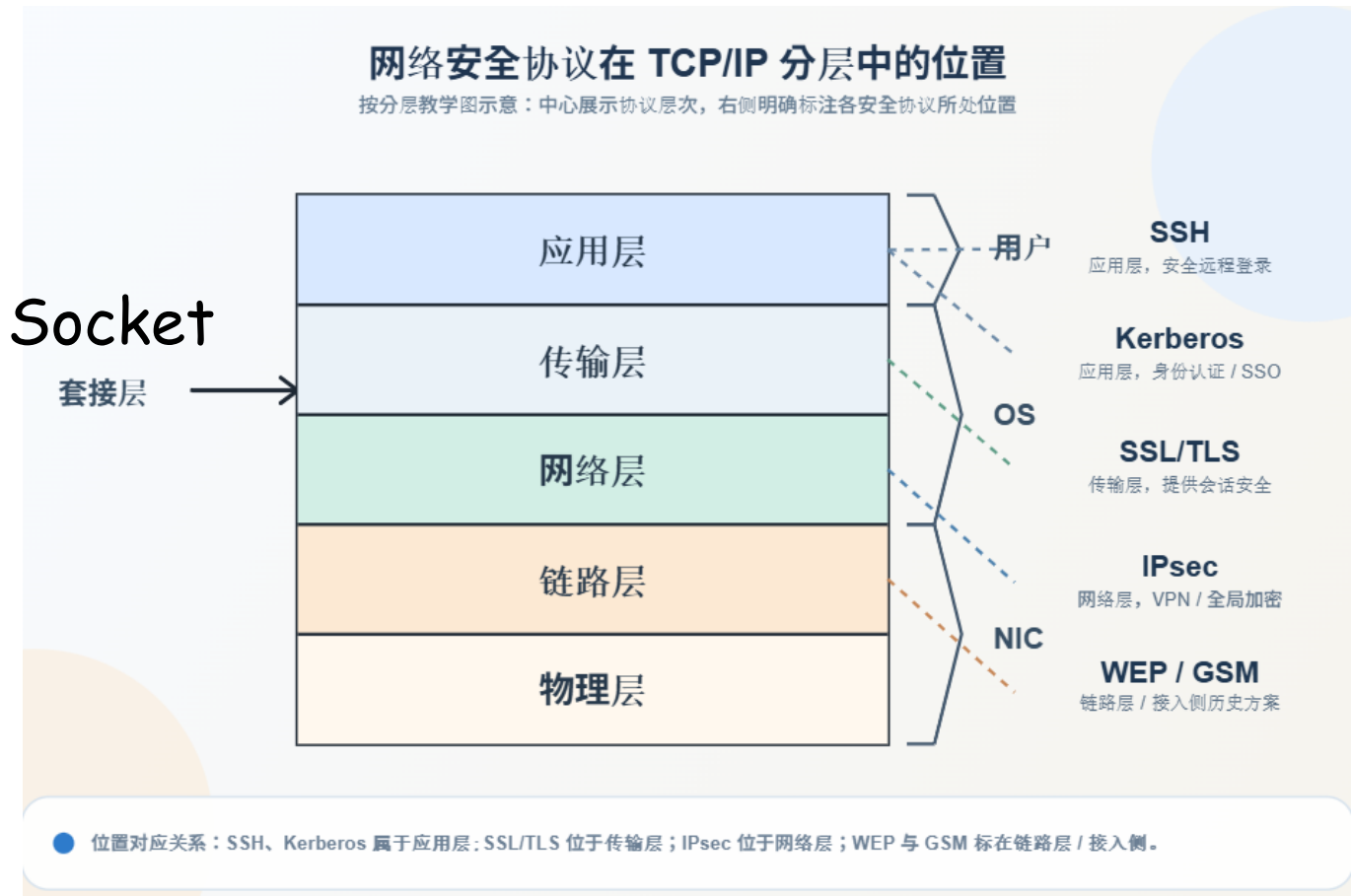
协议 (**Protocols**)

- 简单安全协议
- 现实安全协议

现实安全协议

- 接下来，我们来看实际协议。
 - **SSH——相对简单且实用的协议**
 - **SSL —— Web上的实用安全**
 - **IPSec —— IP层安全**
 - Kerberos —— 对称密钥，单点登录
 - WEK——安全协议的“瑞士奶酪”
 - GSM——移动电话（不）安全

现实安全协议



现实安全协议

- 接下来，我们来看实际协议。
 - **SSH——相对简单且实用的协议**
 - SSL —— Web上的实用安全
 - IPSec —— IP层安全
 - Kerberos —— 对称密钥，单点登录
 - WEP——安全协议的“瑞士奶酪”
 - GSM——移动电话（不）安全

安全外壳协议 (SSH)

- 安全外壳协议: Secure Shell
 - 主要用来在不安全的网络上, 安全地**远程登录和管理服务器**。
- 核心作用:
 - **安全远程登录** 通过SSH从自己的电脑连接到服务器
 - **数据加密传输** SSH对通信内容进行加密
 - **身份认证** 支持多种认证方式, (用户名+密码) (公钥/私钥**)

```
ssh root@192.168.1.1
```

```
ssh -p 2222 root@192.168.1.1
```

安全外壳协议 (SSH)

SSH 连接流程 (简化版)



安全外壳协议 (SSH)

- 创建“安全隧道”

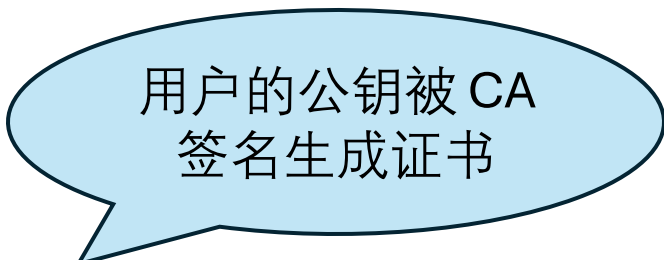
Telnet (1969) → rlogin (1983) → SSH (1995)
明文传输 信任认证 加密隧道

- 通过SSH“隧道”发送的不安全命令随后也会变得安全。

特性	Telnet	rlogin	SSH
是否加密	✗	✗	✓
是否安全	✗	✗ (更差)	✓
登录方式	用户名+密码	信任+密码	密码 / 公钥
使用场景	测试端口	旧Unix环境	现代远程管理
是否推荐	✗	✗	✓

安全外壳协议 (SSH)

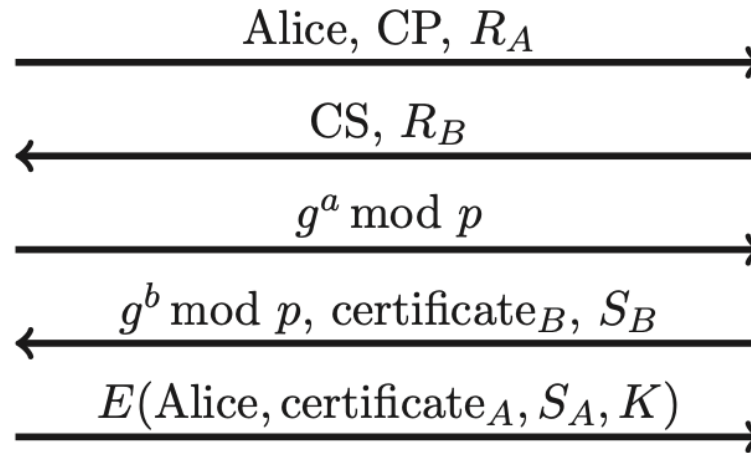
- SSH 身份验证可以基于
 - 用户+密码
 - 公钥 (我信任你这个人)
 - 数字证书 (我信任给你担保的人/机构)
- 这里, 我们考虑**证书**模式
- 我们考虑稍微简化的SSH.....



用户的公钥被 CA
签名生成证书

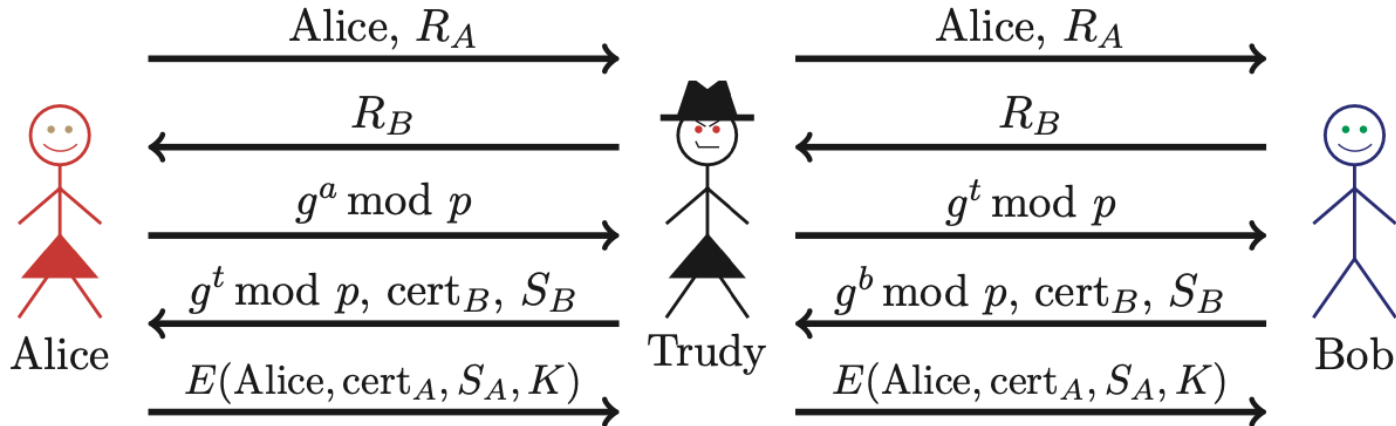
简化版 SSH

协商算法
(灵活性)
DH 密钥交换
(前向安全性)
证书 + 签名
(双向身份认证)
加密传输
(机密性)



- CP = “提议的加密方案”， CS = “选定的加密方案”。
- $H = h(\text{Alice}, \text{Bob}, CP, CS, R_A, R_B, g^a \bmod p, g^b \bmod p, g^{ab} \bmod p)$
- $S_B = [H]_{\text{Bob}}$
- $S_A = [H, \text{Alice}, \text{certificate}_A]_{\text{Alice}}$
- $K = g^{ab} \bmod p$

对 SSH 发起中间人攻击?



• 这次攻击的弱点在哪里? S_B 没有严格绑定到"这次会话 Bob 发出的 DH 值 g^b

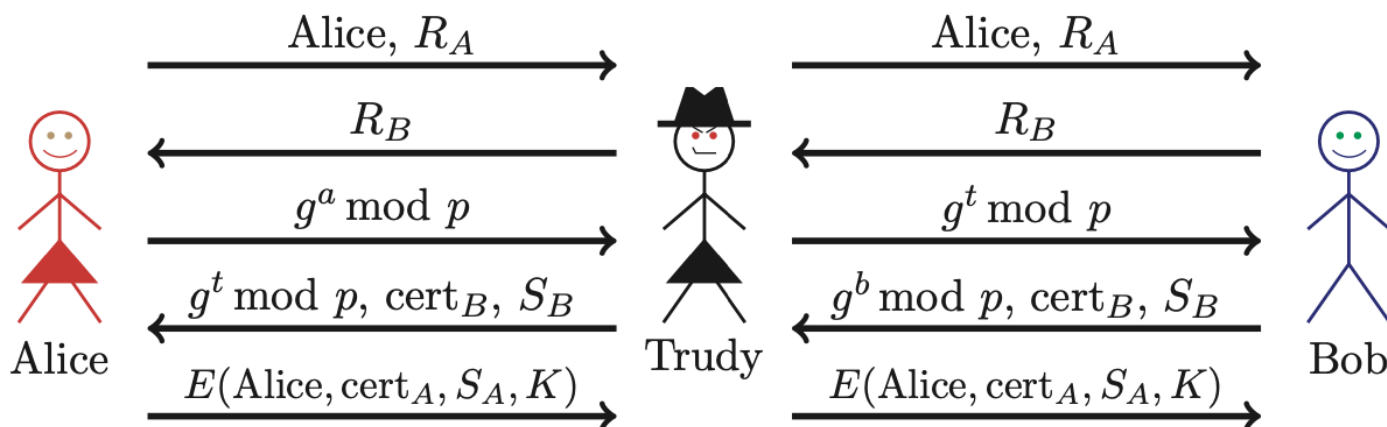
• 爱丽丝计算

$$H_a = h(\text{Alice}, \text{Bob}, \text{CP}, \text{CS}, R_A, R_B, g^a \bmod p, g^t \bmod p, g^{at} \bmod p)$$

• 但鲍勃签了字

$$H_b = h(\text{Alice}, \text{Bob}, \text{CP}, \text{CS}, R_A, R_B, g^t \bmod p, g^b \bmod p, g^{bt} \bmod p)$$

对 SSH 发起中间人攻击?



从外部看:

1. Alice 以为在跟 Bob 建立安全连接
2. Bob 以为在跟 Alice 建立安全连接
3. 双方的证书认证都没有报错

实际上:

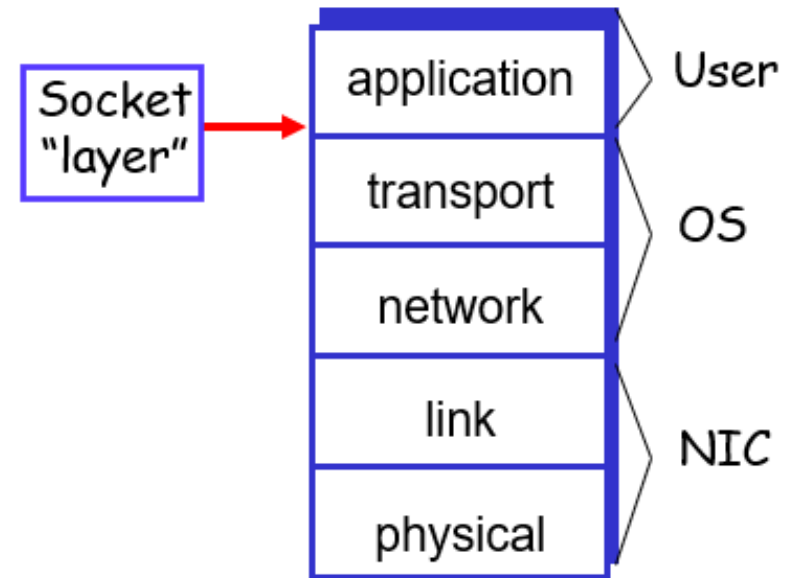
1. Trudy 能解密 Alice 发给 Bob 的所有内容
2. Trudy 能解密 Bob 发给 Alice 的所有内容
3. Trudy 能随意篡改内容, 再重新加密转发

现实中 TLS 1.3 和真实 SSH 都要求把 DH 临时公钥、双方随机数、会话标识符等关键材料全部绑定进签名。

安全套接字层协议 (SSL)

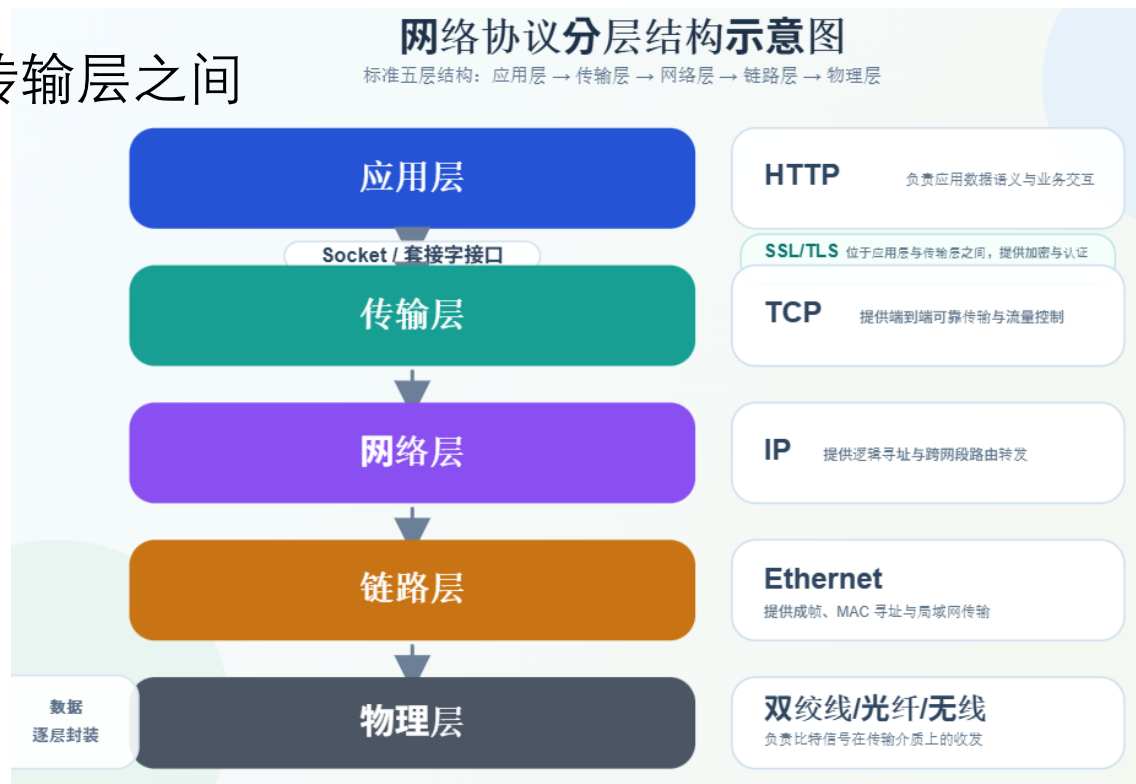


- 安全套接层: Secure Socket Layer
- SSL协议通常用于HTTP和TCP之间。



安全套接字层协议 (SSL)

- 为了更好的了解各个名词:
- HTTP: 在应用层
- Socket: 在应用层和传输层之间
- TCP: 在传输层
- IP: 网络层



安全套接字层协议 (SSL)



- HTTP：应用层协议，负责定义数据内容和通信规则。
- HTTP不安全（明文传输，无身份验证，无完整性保护）
- HTTPS = HTTP + SSL + TCP (利用Socket信息明确接口)

安全套接字层协议 (SSL)

- 访问网页 (输入网址→IP地址, 端口←DNS域名解析)
- Socket (IP: 端口)
- SSL: 完成加密数据
- 通过TCP完成通信连接

客户端 Socket		服务器 Socket
(IP:端口)	⇔	(IP:端口)

客户端: 192.168.1.5:52341
服务器: 93.184.216.34:80

Request URL	https://translate.google.com/_/TranslateWebserverUi/browserinfo?f.sid=-9014071198435479161&bl=boq_translate-webserver_20260422.08_p0&hl=en&soc-app=1&soc-platform=1&soc-device=1&_reqid=262876&rt=j
Request Method	POST
Status Code	● 200 OK
Remote Address	142.251.118.139:443
Referrer Policy	origin

安全套接字层协议 (SSL)

- SSL是现有安全互联网交易使用的协议TLS的前身。

SSL 1.0 (未公开/未正式发布)



SSL 2.0 (1995, 存在严重漏洞)



SSL 3.0 (1996, 较成熟, 但后来被攻破)



TLS 1.0 (1999, IETF标准化, 基于SSL3.0)



TLS 1.1 (2006, 修复已知攻击)



TLS 1.2 (2008, 现代主流长期版本)



TLS 1.3 (2018, 大幅简化+更安全+更快)

SSL2.0: 无法抵抗中间人攻击 密钥设计弱

SSL3.0: 使用不安全加密算法 (RC4)

TLS1.0: SSL3.0升级版, 也不安全

TLS1.1: 修复CBC攻击, 改进初始向量

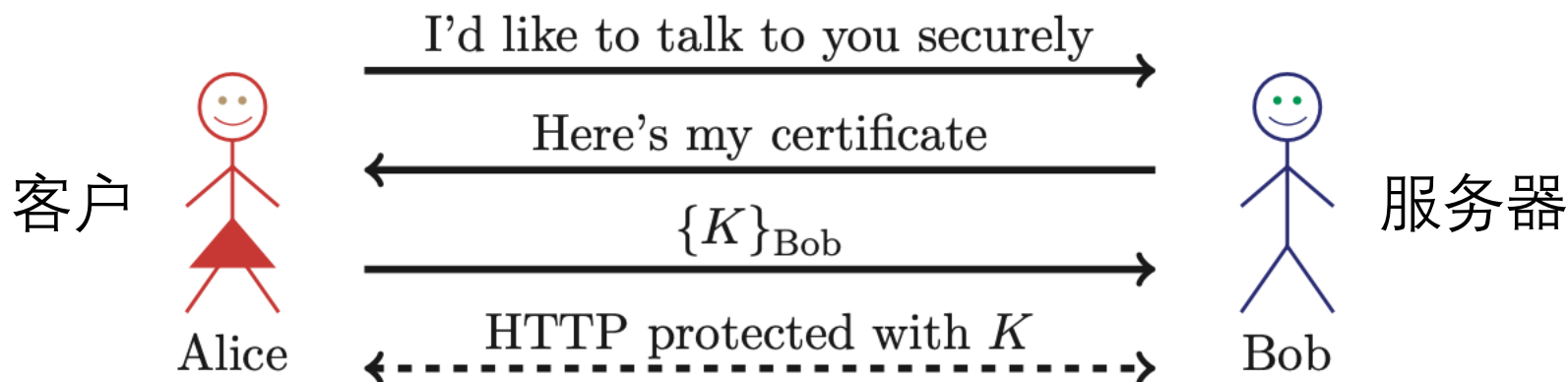
TLS1.2: 长期的主流版本
支持更强的算法 (AES, SHA-256)

TLS1.3: 现代标准

安全套接字层协议 (SSL)

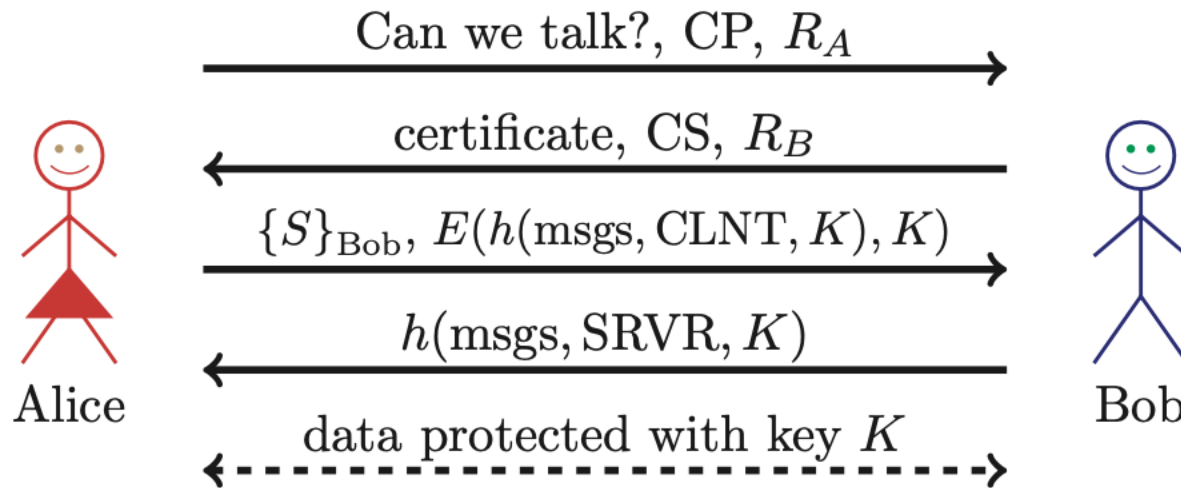
- 例如，如果您想在亚马逊网站上购买一本书.....
 - 您需要确保是在与亚马逊进行交易（**身份验证**）。
 - 发送您的信用卡信息时必须加以保护（**保密性**和**完整性**）。
 - 只要你有钱，亚马逊其实并不在意你是谁.....
 - 所以，无需相互认证。

非常简单的类 SSL 协议



- 爱丽丝确定她是在和鲍勃（服务器）说话吗？
 - 是的，发送了证书
- 鲍勃（服务器）确定他是在和爱丽丝说话吗？
 - 不确定，只有服务端（鲍勃）出示了证书，客户端（爱丽丝）没有出示任何身份凭证。

简化的类 SSL 协议



- S是所谓的**pre-master secret**。
- $K = h(S, R_A, R_B)$
- “msgs”指的是所有之前的消息
- CLNT和SRVR是常数

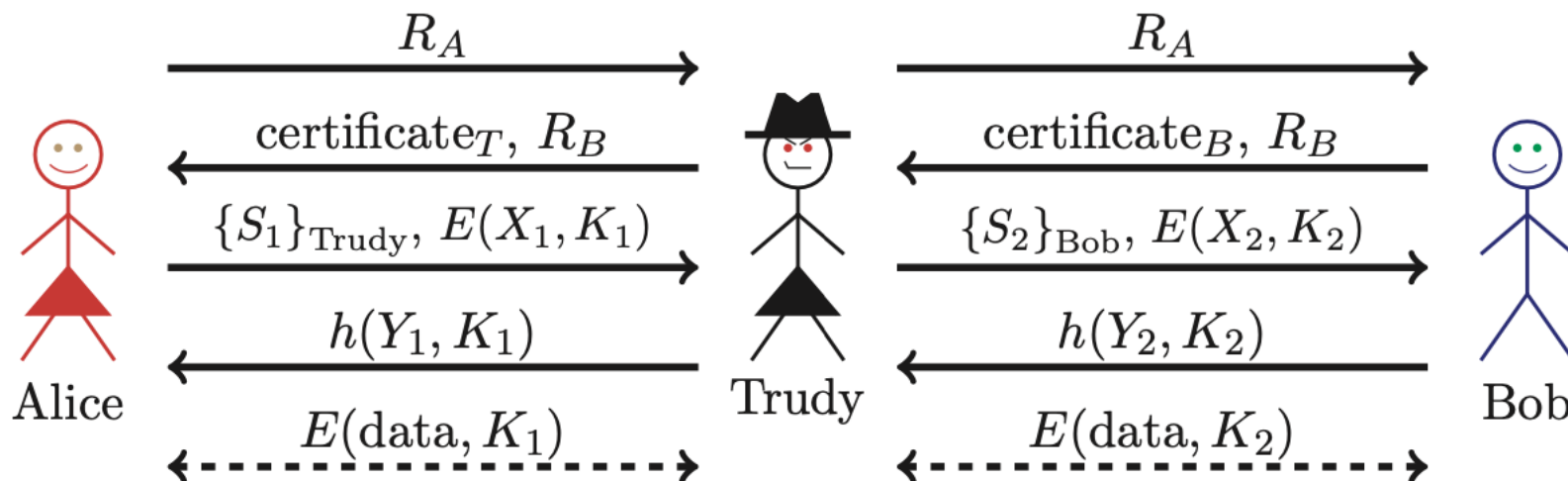
SSL密钥

- $K = h(S, R_A, R_B)$ 这里可以派生出六个密钥
 - 2 个加密密钥：一个加密发送数据，一个加密接收数据
 - 2 个完整性密钥：一个用于保护发送，一个用于保护接收
 - 2 个初始化向量 IV：一个用于发送，一个用于接收
 - 为什么每个方向的密钥都不一样？ **防止重放攻击**
- Q：为什么 $h(msgs, CLNT, K)$ 需要加密？
- A：显然，它并没有增加任何安全性..... **(协议漏洞)**

SSL 身份验证

- Alice负责验证Bob的身份，反之则不然。
 - 客户端如何对服务器进行身份验证？ **证书**
 - 为什么服务器没有验证客户端身份？
- 可以进行双向认证： Bob在消息 2中发送**证书请求**。
 - 那么客户端必须拥有有效的证书
 - 但是，如果服务器想要验证客户端身份，服务器可以改为要求客户端输入密码。

SSL中间人攻击



- **问：** 什么机制可以阻止这种中间人攻击？
- **答：** Bob的证书必须由证书颁发机构（CA）签署。
- 如果签名无效，浏览器会怎么做？
- 当浏览器发出警告时，用户该怎么做？

SSL中间人攻击

- 访问某个网站时，出现此类警告
- 这个警告就是建立访问链接时，后端进行SSL安全验证，
 - 证书验证无效：
 - 证书过期
 - 证书域名不匹配
 - 证书颁发机构（CA）不受信任
 - 网站部署配置错误
 - 使用了 HTTP



Your connection is not private

Attackers might be trying to steal your information from **expired.badssl.com** (for example, passwords, messages, or credit cards). [Learn more about this warning](#)

NET::ERR_CERT_DATE_INVALID



[Turn on enhanced protection](#) to get Chrome's highest level of security

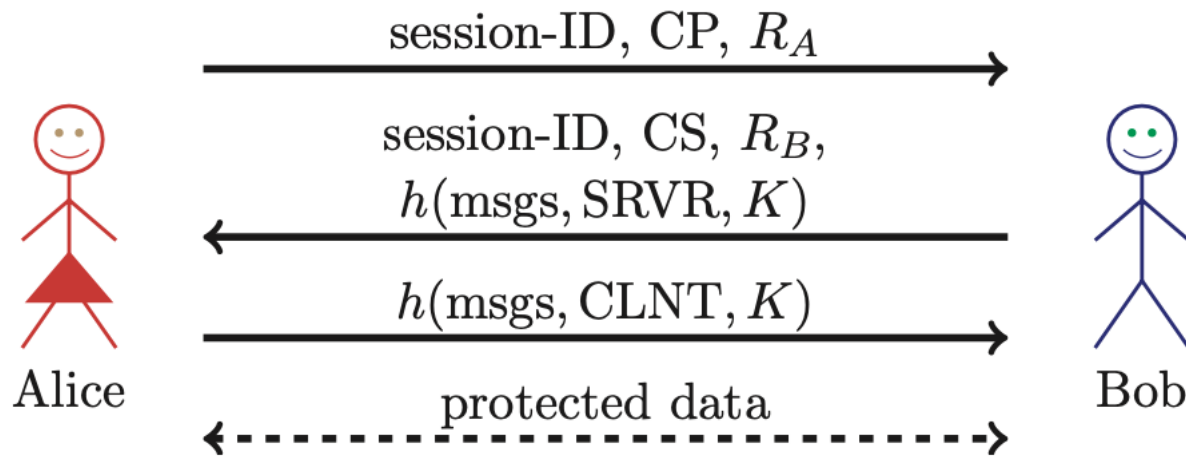
Advanced

Back to safety

SSL会话与连接

- SSL最初由Netscape开发，专门在Web浏览器上使用
 - SSL 专为 HTTP 1.0 设计
- HTTP 1.0 每个会话独立连接，
 - 为提高效率会建立多个并行链接
- 问题：
 - SSL握手成本高，公钥操作需要耗费大量资源。
- 解决方案：
 - SSL支持会话复用，新连接可以复用已有安全会话，只共享对称密钥K。

SSL 连接



- 假设 SSL会话存在
- 所以，S (session-ID) 已经被Alice和Bob认识了。
- 双方都必须记住会话 ID
- 再次， $K = h(S, R_A, R_B)$
- 不再重复进行公钥操作! (依赖于已知的S)

SSL vs IPSec

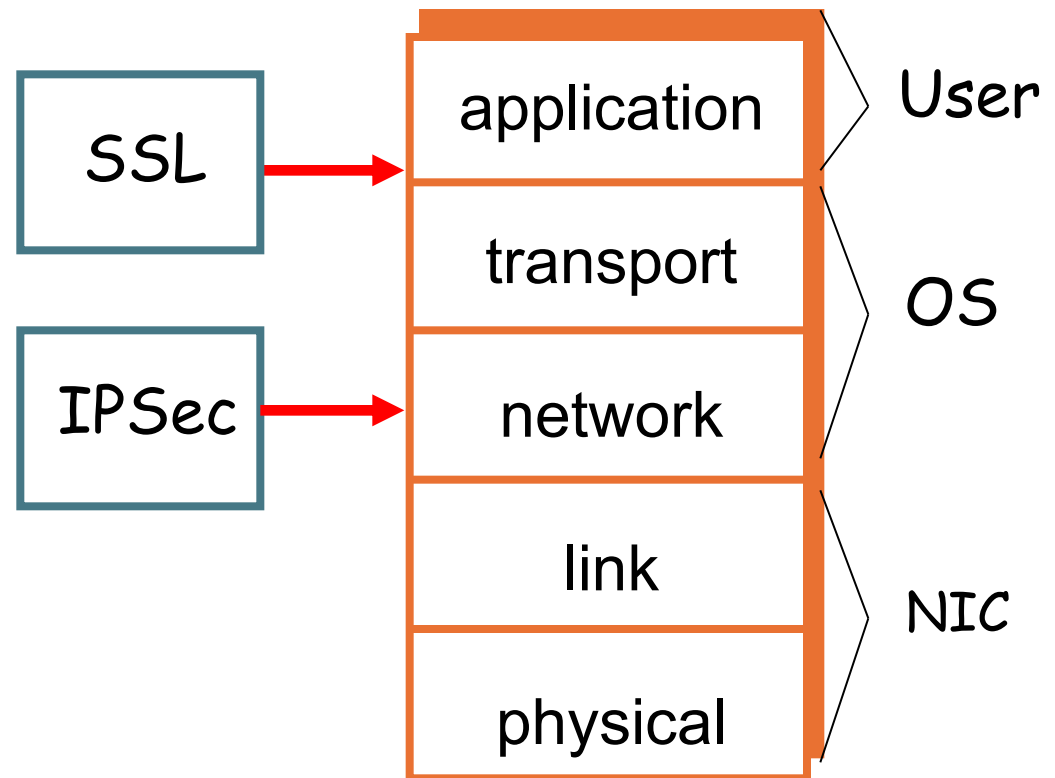
- IPSec——将在下一节讨论
 - 位于**网络层**（操作系统的一部分）
 - 加密、完整性、身份验证等。
 - 过于复杂，存在一些安全“问题”。
- SSL（以及称为TLS的IEEE标准）
 - 位于**套接字层**（用户/应用程序空间的一部分）
 - 加密、完整性、身份验证等。
 - 相对简单而优雅规范

SSL vs IPSec

- IPSec: 操作系统必须知晓, 但应用程序不必知晓。
- SSL: 应用程序必须感知, 但操作系统不必。
- SSL 很早就内置于 Web 中 (Netscape)
- IPSec 常用于 **VPN (安全隧道)** 中
- 不愿对程序进行改造以适应 SSL。
- IPSec 尚未广泛部署 (复杂性等原因) 。
- 结论是什么?
- **互联网的安全性还有待提高!**

IPSec

- IPSec 存在于网络层。
- 优点：IPSec 对应用程序是透明的。



IPSec与复杂性

- IPSec 是一种复杂的协议
- 过度设计
 - 很多（通常无用的）功能
- 有缺陷的 一些重大的安全漏洞
- 互操作性可能是一个挑战
 - 这完全违背了制定标准的初衷！
- 复杂的
- 而且，我有没有提到，这很复杂？

规格说明书分为三部分：

RFC2407

RFC2408

RFC2409

并且属于不同的组织

<https://datatracker.ietf.org/doc/html/rfc2407>

IPSec

- IPSec分为两部分.....
- **IKE:** 互联网密钥交换
 - 相互认证
 - 建立会话密钥
 - 两个“阶段”——例如 SSL 会话/连接
- **ESP/AH** Encapsulating Security Payload
 - **ESP** : 封装安全有效载荷——用于保密性和/或完整性
 - **AH** : 身份验证标头——仅完整性
Authentication Header

IKE

- IKE分为两个阶段
 - 第一阶段—**IKE** 安全关联
 - 第二阶段——**IPSec**安全关联
- 第一阶段类似于SSL **会话**
- 第二阶段类似于 SSL **连接**
- IKE中似乎并不需要两个阶段。
 - 在IPSec的背景下，这意味着
- 如果不能进行多次第二阶段，那么进行两阶段的成本会**更高！**

IKE第一阶段

- 4 种不同的“密钥选项”
 - 公钥加密（原始版本）
 - 公钥加密（改进版）
 - 公钥签名
 - 对称密钥
- 对于每一种情况，都有 2 种不同的“模式”。
 - 主模式和激进模式
- **IKE 第一阶段共有 8 个版本！**
- 还需要更多证据证明它是过度设计吗？

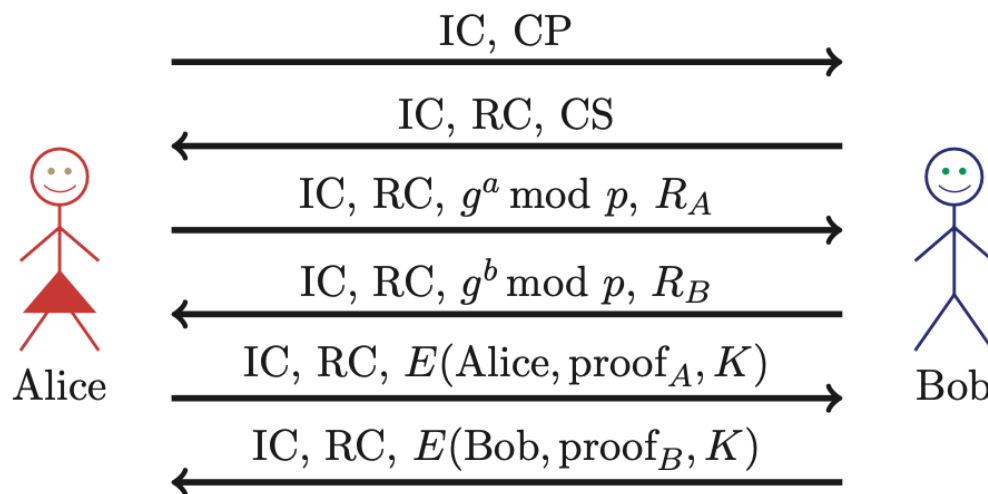
IKE第一阶段

- 我们讨论了8个第一阶段密钥中的6个。
 - 公钥签名（主要模式和激进模式）
 - 对称密钥（主模式和攻击模式）
 - 公钥加密（主要和激进）
- 为什么需要公钥加密和公钥签名？
 - 总是知道自己的私钥
 - **可能**（最初）不知道对方的公钥

IKE第一阶段

- 使用临时Diffie -Hellman密钥交换协议建立会话密钥
 - 提供完美前向保密 (PFS)
- 设 a 为 Alice 的Diffie -Hellman 指数
- 设 b 为 Bob 的Diffie -Hellman 指数
- 设 g 为生成元, p 是素数
- 请注意, p 和 g 是公共的。

IKE 第一阶段：数字签名（主模式）



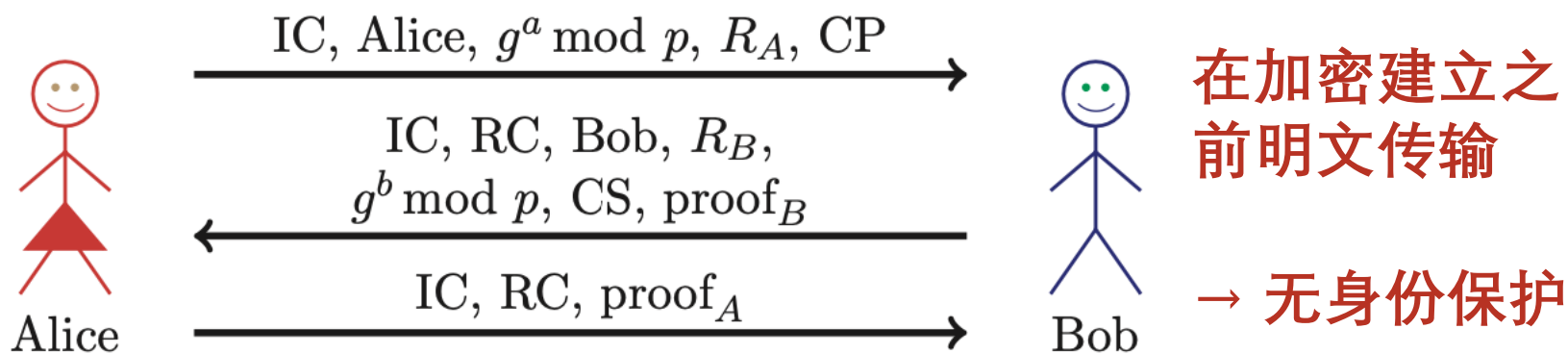
参数协商

→ DH 交换

→ 身份认证

- CP = 提议的加密方案，CS = 选择出的加密方案。
- IC = 发起方“cookie”，RC = 响应方“cookie”
- $K = h(IC, RC, g^{ab} \bmod p, R_A, R_B)$
- $SKEYID = h(R_A, R_B, g^{ab} \bmod p)$
- $Proof_A = [h(SKEYID, g^a \bmod p, g^b \bmod p, IC, RC, CP, "Alice"))]_{Alice}$

IKE 第一阶段：数字签名（激进模式）



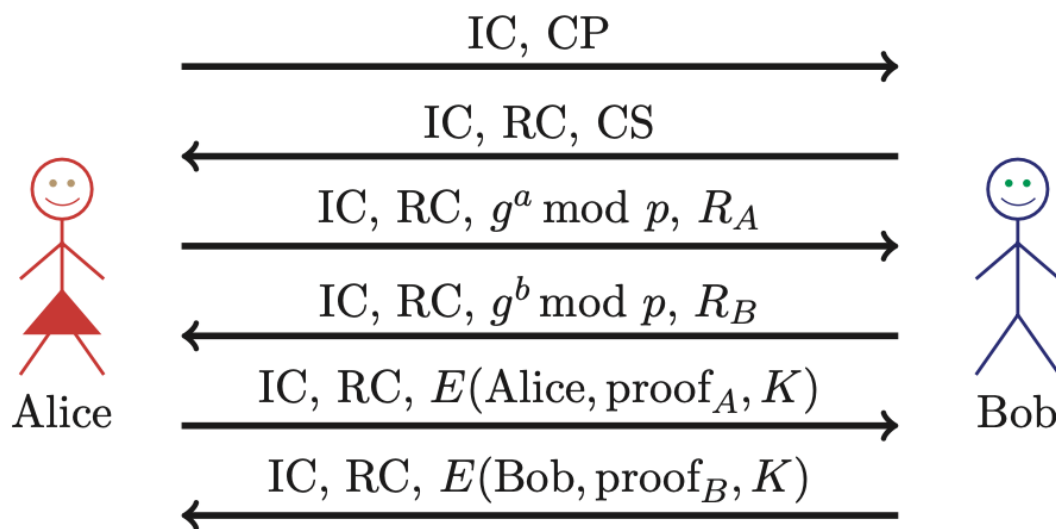
- 与主模式的主要区别
 - 并非试图隐藏身份
 - 无法协商 g 或 p

对比项	主模式	激进模式
消息数	6 条	3 条
速度	慢	快
身份保护	✅ 身份加密传输	❌ 身份明文可见
适用场景	固定 IP 的站点间 VPN	动态 IP 的远程接入 VPN
抗中间人	强	较弱（身份暴露）

主模式 vs 激进模式

- | | 激进模式 | 主模式 |
|-------|---------|---------------|
| 被动攻击者 | ✗ 能看到身份 | ✓ 身份已加密，看不到 |
| 主动攻击者 | ✗ 能看到身份 | ✗ 也能确定身份 (见下) |
- 主模式**必须**实现。
 - 激进模式**应该**启用。
 - **不实现是允许的，但你需要有充分理由，否则就是偷懒。**
 - 可能会造成互操作性问题
 - 用于公钥签名认证
 - **被动攻击者**在攻击模式下知道 Alice 和 Bob 的身份，但在主模式下则不知道。
 - **主动攻击者**可以在主模式下确定 Alice 和 Bob 的身份。

IKE 第一阶段：对称密钥（主模式）



- 与签名模式相同，但.....
 - K_{AB} = 预先共享的对称密钥
 - $K = h(IC, RC, g^{ab} \bmod p, R_A, R_B, K_{AB})$
 - $SKEYID = h(K, g^{ab} \bmod p)$
 - $Proof_A = h(SKEYID, g^a \bmod p, g^b \bmod p, IC, RC, CP, "Alice")$

对称密钥（主模式）的问题

- Catch-22 死循环

Alice 第5条消息用密钥 K 加密了身份



Bob 要解密身份，必须先有 K



K 由 K_{AB} （预共享密钥）派生



要找到正确的 K_{AB} ，Bob 必须先知道“自己在和 Alice 说话”



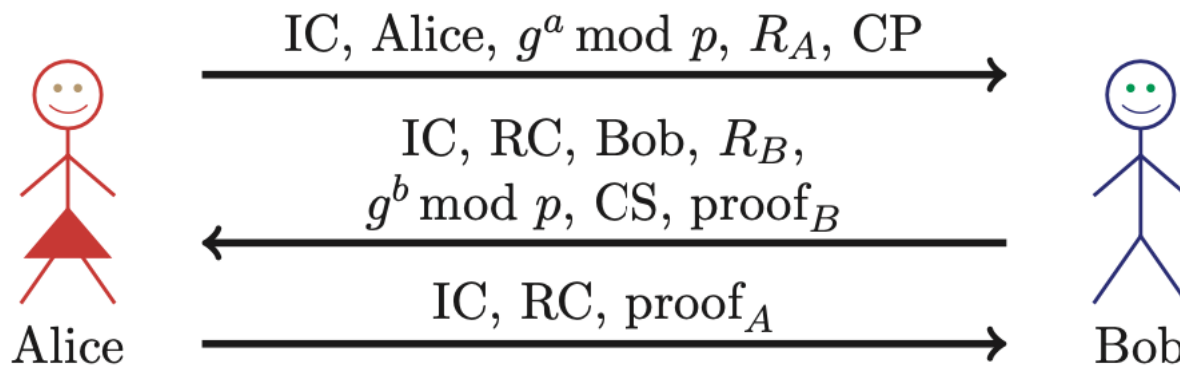
要知道对方是 Alice，必须先解密身份



回到起点 ↑

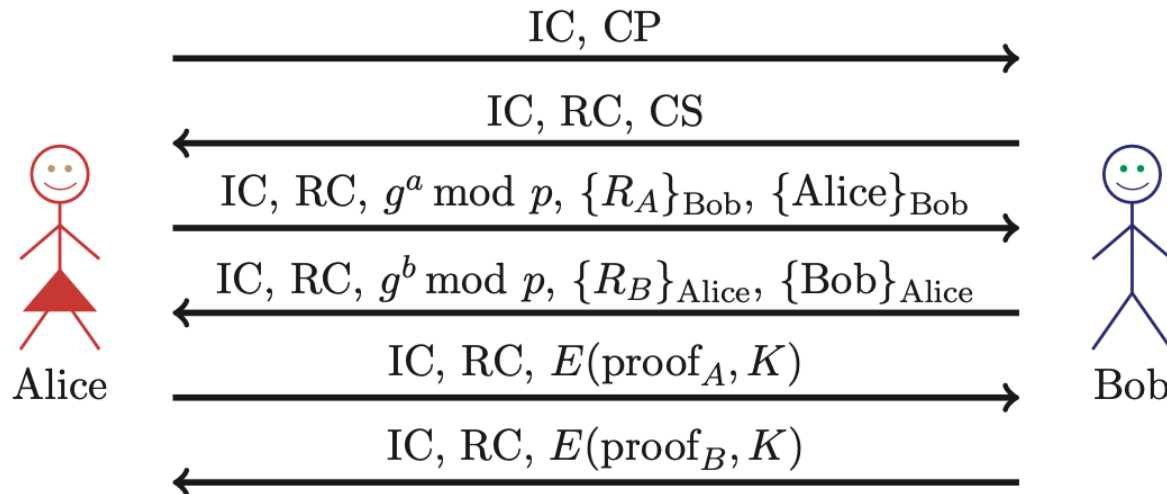
- 解决方案： **使用Alice的IP地址作为ID!** IP在网络层暴露
 - 缺陷：对于“空中飞人来说”毫无用处。 **移动用户**
 - 缺陷：为什么要费尽心思在 6 消息协议中隐藏身份呢？

IKE 第一阶段：对称密钥（激进模式）



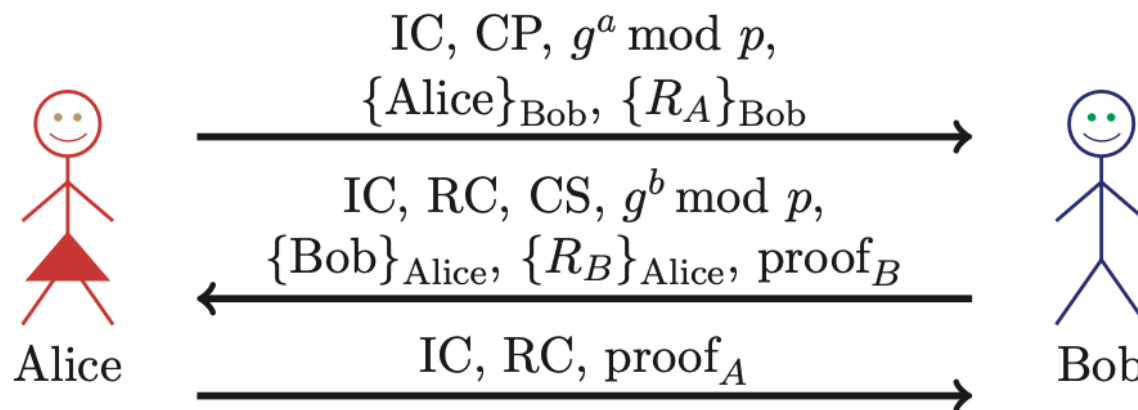
- 与数字签名激进模式相同的格式
- 并非试图隐瞒身份.....
- 因此，主模式**不存在问题**。
- 主模式没有隐藏身份

IKE 第一阶段：公钥加密（主模式）



- CP = 提议的加密方案，CS = 选择出的加密方案。
- IC = 发起方“cookie”，RC = 响应方“cookie”
- $K = h(IC, RC, g^{ab} \bmod p, R_A, R_B)$
- $SKEYID = h(R_A, R_B, g^{ab} \bmod p)$
- $\text{Proof}_A = h(SKEYID, g^a \bmod p, g^b \bmod p, IC, RC, CP, \text{“Alice”})$

IKE 第一阶段：公钥加密（激进模式）

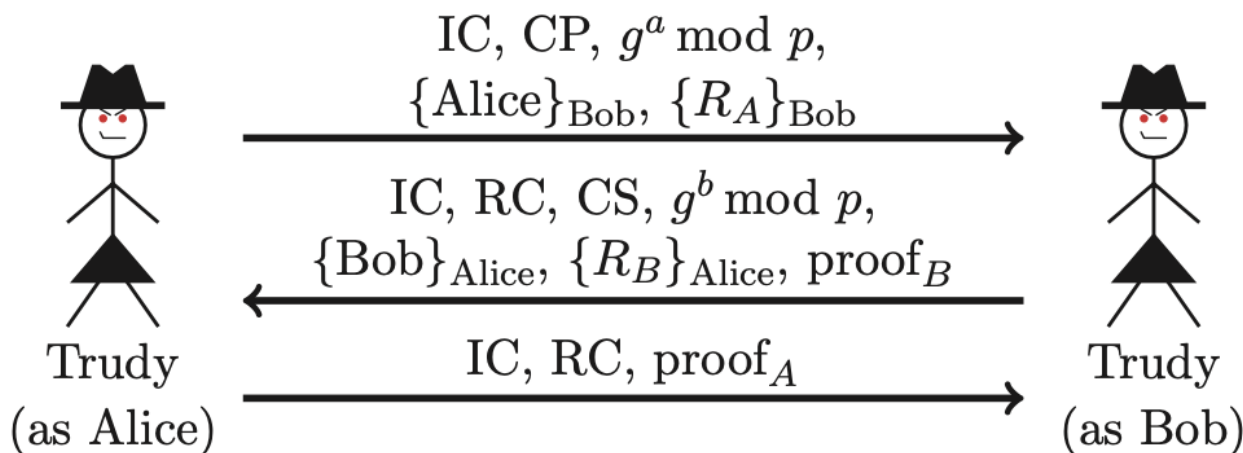


- K , $Proof_A$, $Proof_B$ 计算方式与主模式相同
- 请注意，身份信息已隐藏。
 - 这是唯一一种用于隐藏身份的激进模式。
 - 那么，为什么还需要一个主模式呢？

公钥加密问题

- 在公钥加密中，激进模式.....
- 假设 **Trudy** 生成
 - 指数 **a** 和 **b**
 - 随机数 **R_A** 和 **R_B**
- Trudy 可以计算“有效”密钥和证明: **$g^{ab} \bmod p$** 、**K**、**SKEYID**、**Proof_A** 和 **Proof_B**
- 所有这些在主模式下也有效。

公钥加密问题



- Trudy can forge information that appears to be between Alice and Bob.
- For any observer, **including Alice and Bob, this seems to be effective!**

合理否认

- 特鲁迪可以伪造一段看似是爱丽丝和鲍勃之间的“对话”。
 - 就连爱丽丝和鲍勃也觉得这说法有道理!
- 安全 **失败**? ?
- 在IPSec公钥选项中，它是一个 **特征...**
 - **合理的否认**：爱丽丝和鲍勃可以否认发生过任何对话!
- 在某些情况下，这可能会造成问题。
 - 例如，如果爱丽丝从鲍勃那里购买了一件商品，她之后可以反悔（除非她签了字）。

IKE 第一阶段“Cookie”

- IC和RC ——cookie
（或“防堵塞令牌”）旨在防止拒绝服务攻击
 - 与网络 **Cookie** 无关
- 为了减少拒绝服务攻击的威胁，Bob 希望尽可能长时间地保持“**无状态**”状态。
- 但鲍勃必须记住第一条消息中的CP
（第六条消息中需要用到它来证明身份）。
- Bob必须从第一条消息开始保持状态
 - 因此，**这些“cookie”**几乎无法提供任何DoS防护。

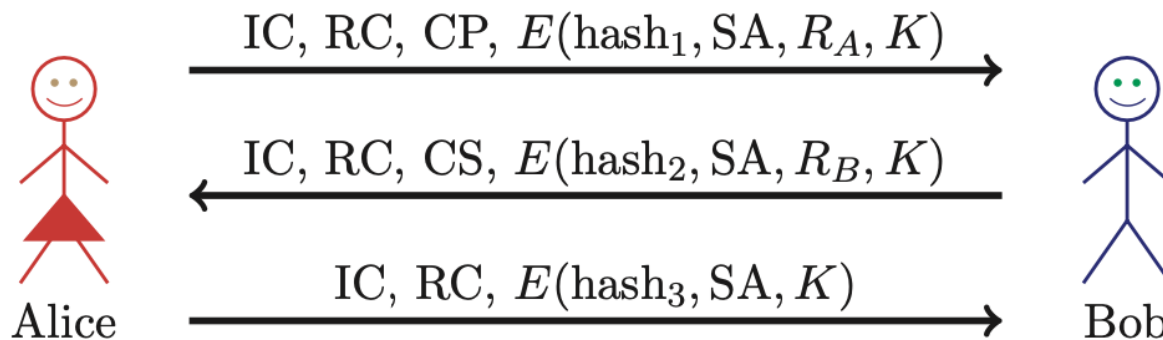
IKE第一阶段总结

- IKE第一阶段的结果是：
 - 相互认证
 - 共享对称密钥
 - IKE安全协会 (SA: Security Association)
- 但第一阶段成本很高。
 - 尤其是在公钥和/或主模式下
- IKE 的开发者认为它会被用于很多方面——而不仅仅是 IPSec。
 - 这在一定程度上解释了过度设计的原因.....

IKE第二阶段

- 在使用IKE 第二阶段之前，必须完成IKE 第一阶段
- 这种情况下：
 - 共享对称密钥K，IPSec cookie IC 和RC，IKE 安全关联都已经建立并且已知。
- 与 SSL 的比较.....
 - SSL 会话与 IKE 第一阶段类似
 - SSL 连接类似于 IKE 第二阶段。
- IKE可以用于很多事情，但实际上却并非**如此！**

IKE第二阶段



- 密钥 K 、 IC 、 RC 和 SA 已在第一阶段中确定。
- 保密提案 CP 包括 ESP 和/或 AH
- 哈希值 1、2、3 取决于 $SKEYID$ 、 SA 和 R_A 。和 R_B
- 从 $KEYMAT = h(SKEYID, R_A, R_B, \text{junk})$ 派生的密钥
- 回想一下, $SKEYID$ 取决于第一阶段密钥方法
- 可选的 PFS (临时Diffie -Hellman 交换)

IPSec

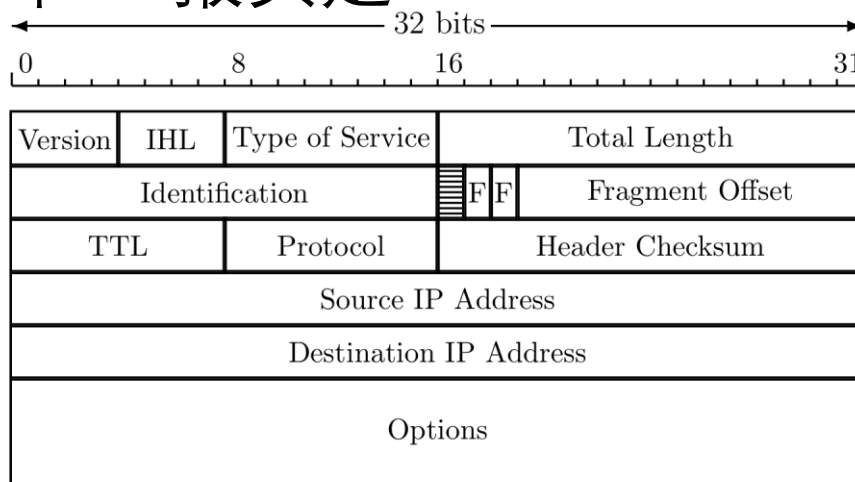
- IKE 第一阶段结束后，我们有一个 IKE SA。
- IKE 第二阶段之后，我们建立了一个 IPSec SA。
- 身份验证已完成，并已获得共享对称密钥（会话密钥）。
- 接下来怎么办？
 - 我们希望保护**IP数据报**。
 - 但什么是IP数据报？
 - 从IPSec的角度来看.....

IP数据报

□ IP 数据报的形式为

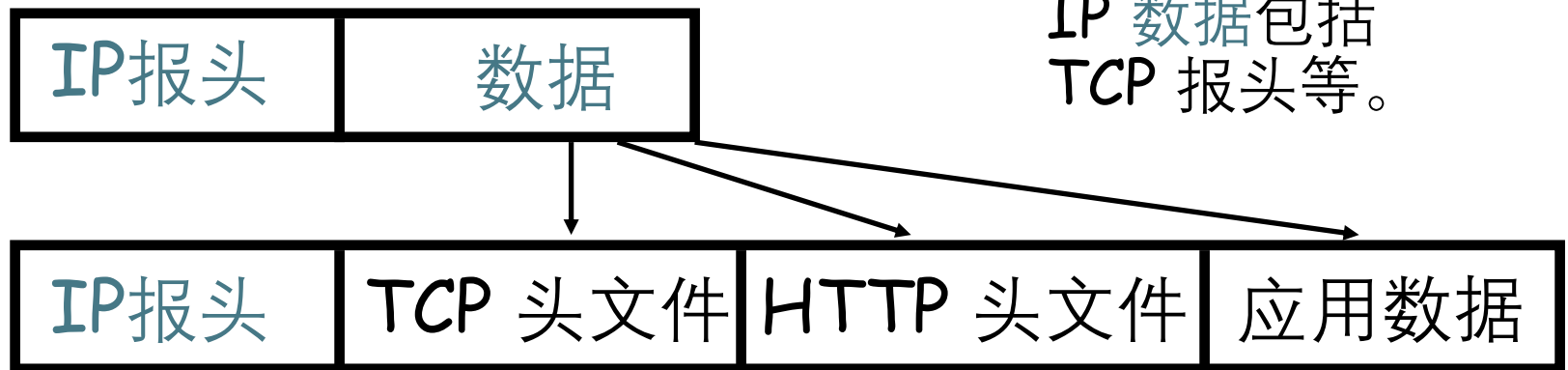


• 其中 IP 报头是



IP和TCP

- 例如，考虑一下网络流量。
 - IP封装了TCP和.....
 - TCP封装了HTTP



IPSec传输模式

- IPSec传输模式



- 专门为主机间传输设计的传输模式

- 运输方式高效

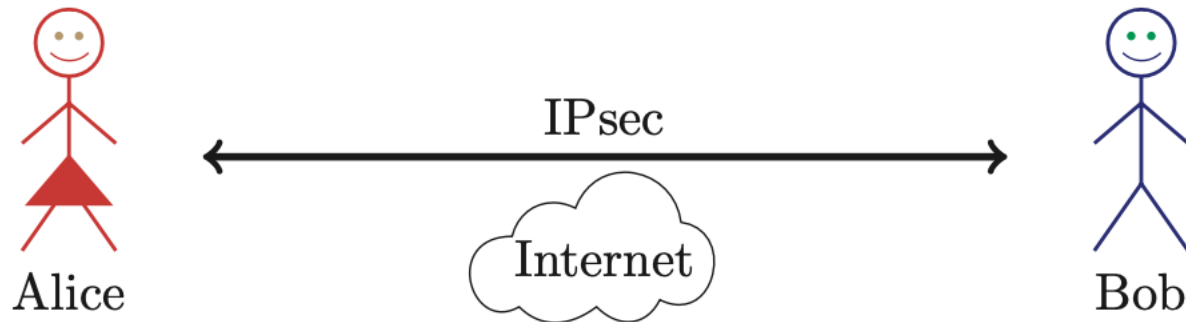
- 仅添加少量额外头部信息

- 原始标题保持不变

- 被动攻击者可以看到是谁在说话

IPSec: 主机到主机

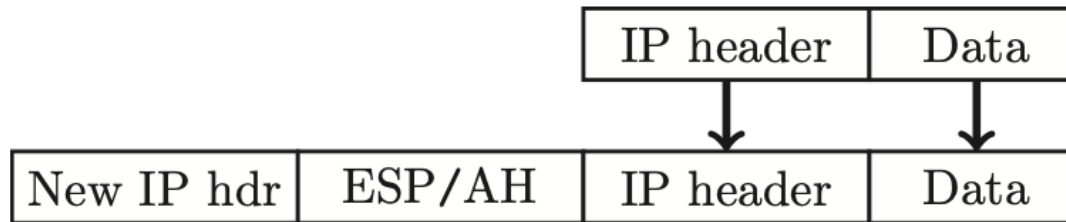
- 此处使用IPSec传输模式



- 两者之间可能存在防火墙。
 - 如果是这样, 那会是个问题吗?

IPSec隧道模式

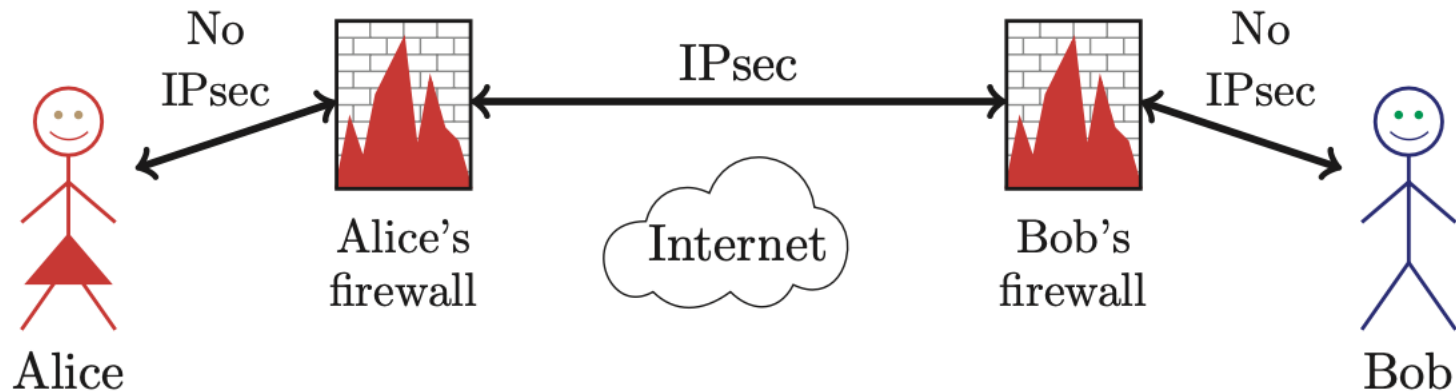
• IPSec隧道模式



- ❑ 防火墙到防火墙的隧道模式 交通
- ❑ 封装在 IPSec 中的原始 IP 数据包
- ❑ 攻击者无法看到原始 IP 报头。
 - 从防火墙到防火墙的新 IP 标头
 - 攻击者并不知道哪些主机正在通信。

IPSec: 防火墙到防火墙

- 此处使用IPSec隧道模式



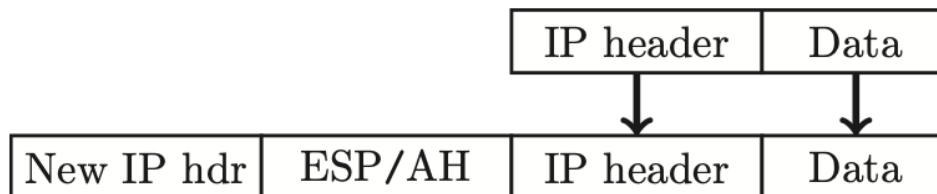
- ❑ 注意:本地网络未受保护
- ❑ 有什么优势吗?

IPSec模式比较

• 运输模式



• 隧道模式



- ❑ 运输模式
 - 主机对主机
- ❑ 隧道模式
 - 防火墙到防火墙
- ❑ 运输模式并非必要条件.....
- ❑ 但它的效率更高。

IPSec 安全

- 什么样的保护措施?
 - 保密性?
 - 完整性?
 - 两个都?
- 要保护什么?
 - 数据?
 - 标题?
 - 两个都?
- ESP/AH 允许这些的一些组合。

AH vs ESP

• AH —— 身份验证标头

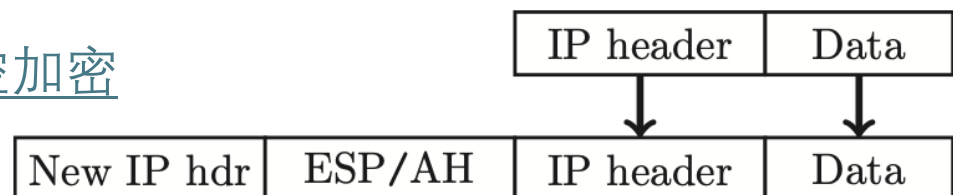
- 仅限完整性 (不保密)

- 对 IP 报头之外的所有内容以及报头中的某些字段
(为什么不是所有字段?) 进行完整性保护。



• ESP —— 封装安全有效载荷

- 完整性和保密性都是必需的。
- 保护 IP 报头之外的所有内容
- 仅保护完整性时, 使用空加密



IPSec 安全

• 什么样的保护措施?

- 保密性?
- 完整性?
- 两个都?

• 要保护什么?

- 数据?
- 标题?
- 两个都?

• ESP/AH 允许这些的一些组合。

保护类型	含义	对应机制
保密性 (Confidentiality)	数据不被窃听, 内容加密	ESP 加密
完整性 (Integrity)	数据不被篡改, 来源可验证	AH / ESP 认证
两个都要	同时加密 + 认证	ESP (含认证)

保护范围	含义	对应模式
数据 (Payload)	只保护 IP 包的数据部分	传输模式 (Transport Mode)
标题 (Header)	连 IP 头 (源/目的地址等) 也保护	隧道模式 (Tunnel Mode)
两个都要	整个原始 IP 包全部保护	隧道模式 (将原包完整封装)

协议	加密	认证/完整性	保护 IP 头
AH (Authentication Header)	X	✓	✓ (含外层头)
ESP (Encapsulating Security Payload)	✓	可选 ✓	X (不保护外层头)
ESP + AH 组合	✓	✓	✓

ESP 空加密

- 空加密: 对于任意P和密钥K, $\text{Null}(P,K) = P$ 。 没有加密
- 使用 NULL 加密的 ESP 与 AH 相同吗 ?

对比项	AH	ESP (NULL 加密 + 认证)
加密	无	无 (NULL)
认证/完整性	✓	✓
保护 IP 外层头	✓	X
保护范围	几乎整个包含头部	仅 Payload 部分

- 区别: AH 认证覆盖 IP 头 (包括源/目的地址), 而 ESP 即使用 NULL 加密也不保护外层 IP 头。因此两者在功能上并不等价, ESP+NULL 无法替代 AH 对 IP 头的完整性保护。

扩展：网络分层与协议

层	协议 / 功能	封装结构	
应用层	HTTP 应用数据语义与业务交互	[HTTP头 HTTP Body] 原始明文数据	
(会话层)	SSL/TLS 提供加密与身份认证	[TLS记录头 加密(HTTP内容)] HTTP内容被加密封装	Socket
传输层	TCP 端到端可靠传输与流量控制	[TCP头 TLS加密数据] 加入端口号、序列号等	
网络层	IP 逻辑寻址与跨网段路由转发	[IP头(IPv4/IPv6) TCP头 TLS数据] 加入源/目的IP地址	路由器
链路层	Ethernet 成帧、MAC寻址与局域网传输	[Eth头 IP头 TCP头 TLS数据 Eth尾] 加入帧头帧尾	交换机
物理层	双绞线 / 光纤 / 无线 负责比特信号在介质上的收发	010100101101... 比特流信号传输	

发送
封装

接收
解封

Next Week

- Kerberos —— 对称密钥, 单点登录
- WEP —— 安全协议的“瑞士奶酪”
- GSM —— 移动电话 (不) 安全

Thank you.